

Харківський національний університет імені В. Н. Каразіна
Навчально-науковий інститут комп'ютерних наук та штучного інтелекту
Кафедра кібербезпеки інформаційних систем, мереж і технологій

Розрахунково-графічна робота №1

«Згадати все...»

дисципліна: «Крос-платформне програмування»

Виконав: студент групи КБ-21

Коваленко Нікіта

Олександрович

Перевірів: доцент кафедри ІПСІТ

Споров Олександр Євгенович

Розрахунково-графічна робота №1

Мета роботи:

- Згадати основні відомості про ООП, що були вивчені в межах курсу «Об'єктно - орієнтоване програмування»: основні принципи об'єктно - орієнтованого підходу до програмування та дизайну, особливості застосування інтерфейсів та абстрактних класів, основи роботи з колекціями, основи роботи з файлами (операції читання / запису).
- Згадати основи чисельних методів, що були вивчені в межах курсу «Вища математика» та розібрати їх реалізацію на основі інтерфейсів.

Завдання:

Створити програмне рішення, що моделює поняття функції та дозволяє:

- створювати функції, що задані або аналітично (вираз в кодї), або ж таблично;
- обчислювати значення функції для вказаного аргументу;
- числовим методом із вказаною точністю обчислювати похідну заданої функції.

Завдання 1:

Створити програмне рішення для одноманітного представлення функції однієї змінної (тип double), заданої різними способами: аналітичним (формула, що визначає функцію, вказана в тексті програми) та табличним. Чисельно продиференціювати із зазначеною точністю задані таким чином функції однієї змінної. Для розв'язку задачі скористатись рекомендаціями, що наведені нижче, та створити програмне рішення за наведеною в рекомендаціях схемою.

Для тестування програмного рішення обчисліть із заданою точністю на відрізок $x \in [1.5, 6.5]$ з кроком 0.05 для декількох тестових функцій значення функції та її похідної.

Результати диференціювання зберегти в текстових файлах даних. Графічно відобразити збережені дані за допомогою інших стандартних програмних рішень. Для тестування програми використати такі функції:

- перша функція: $f(x) = \exp(-x^2) \sin(x)$;
- друга функція: $f(x) = \exp(-a x^2) \sin(x)$ для заданих значень a ($a = 0.5, 1.0, 1.5$);
- третя функція: функція задана табличним чином — значення незалежної змінної та функції ($\sin(x)$) збережені у текстовому файлі даних. Таблиця значень функції зберігається у відповідному класі у вигляді змінної типу ArrayList.

Завдання 2:

Провести рефакторинг створеної системи.

Завдання 3:

Провести реінжиніринг програмного рішення, додавши можливості:

- дані для функції, що задана табличним способом, отримуються із CSV-файлу;
- дані для функції, що задана табличним способом, зберігаються в структурі типу TreeSet;
- дані для функції, що задана табличним способом, зберігаються в структурі типу TreeMap;
- функція, що задана аналітично, може бути зазначена в процесі роботи програми у вигляді рядка. Для такого варіанту завдання функції похідна має бути обчислена як чисельним, так і символьним способом. Передбачити можливість завдання функції без параметрів (наприклад, $f(x) = \exp(-x) \cdot \sin(x)$) та з одним параметром (наприклад, $f(x) = \exp(-ax) \cdot \sin(x)$);
- створити програму з графічним інтерфейсом користувача для відображення аналітично заданої функції та її похідної у зазначених межах.
- додати можливості параметризації — для того, щоб обчислення можна було виконати для функцій різних типів.

Хід роботи:

Першим кроком було створення програми для реалізації початкових вимог (див. Додаток А), результатом роботи якої є 5 файлів з розрахованими значеннями тестових функцій при відповідних аргументах та автоматично згенерований файл табличної функції для тестування.

Нижче (див. Рисунок 1-5) подано графіки, створені за розрахованими програмою даними.

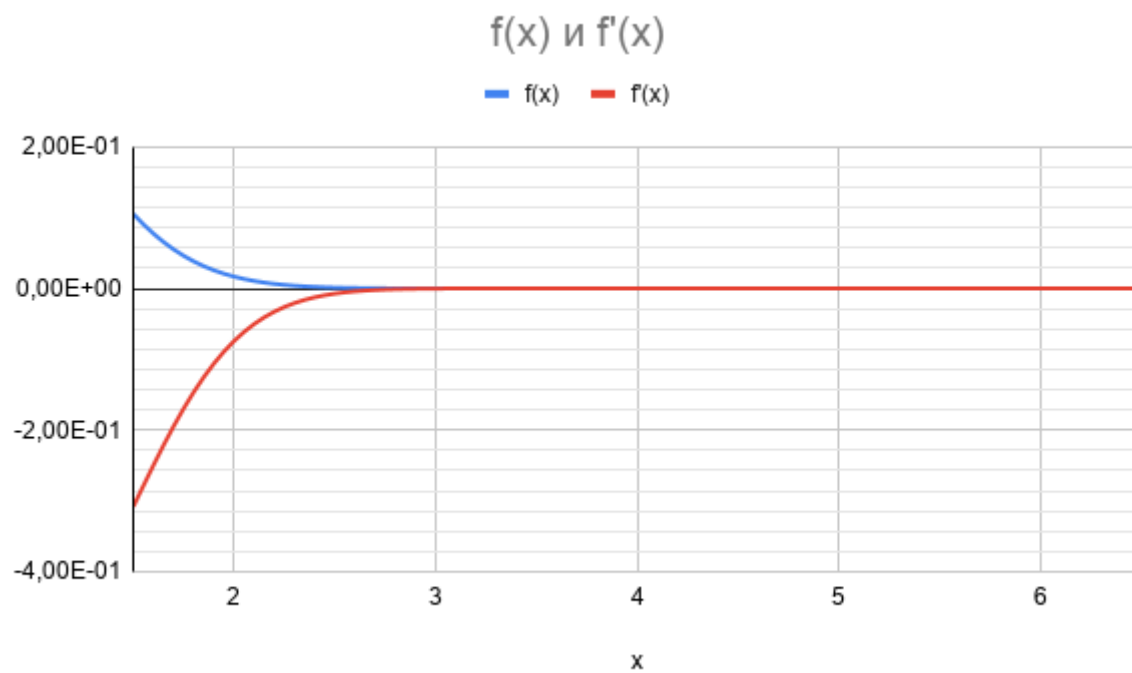


Рисунок 1 – Графік результатів функції 1.

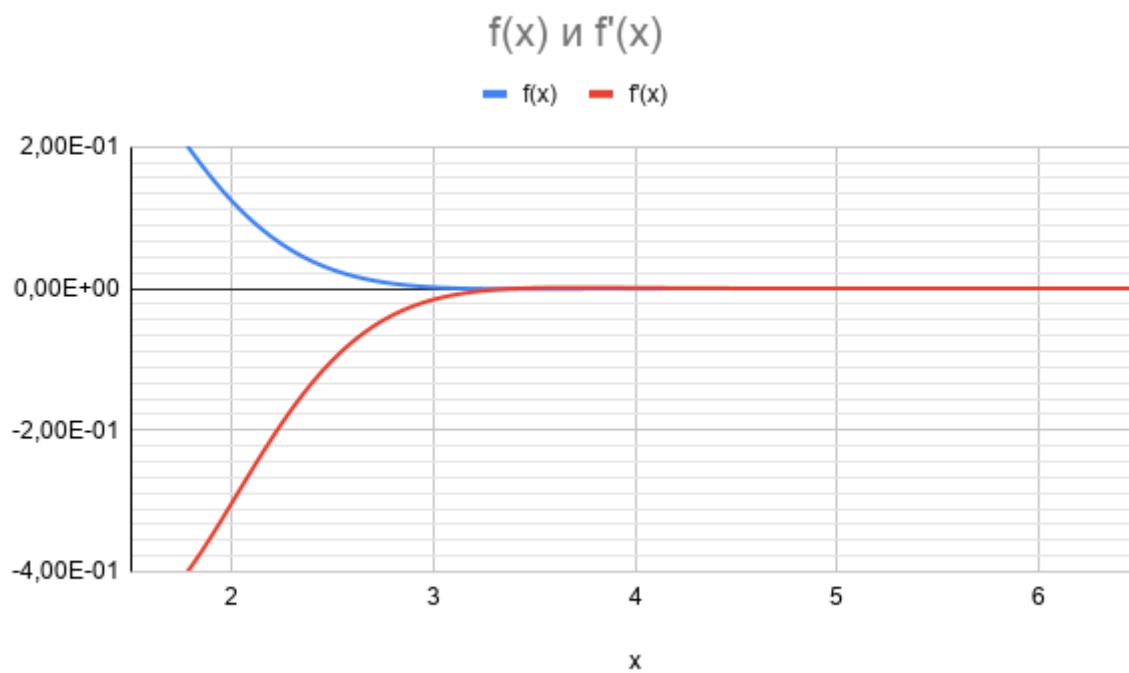


Рисунок 2 – Графік результатів функції 2 при $a=0,5$.

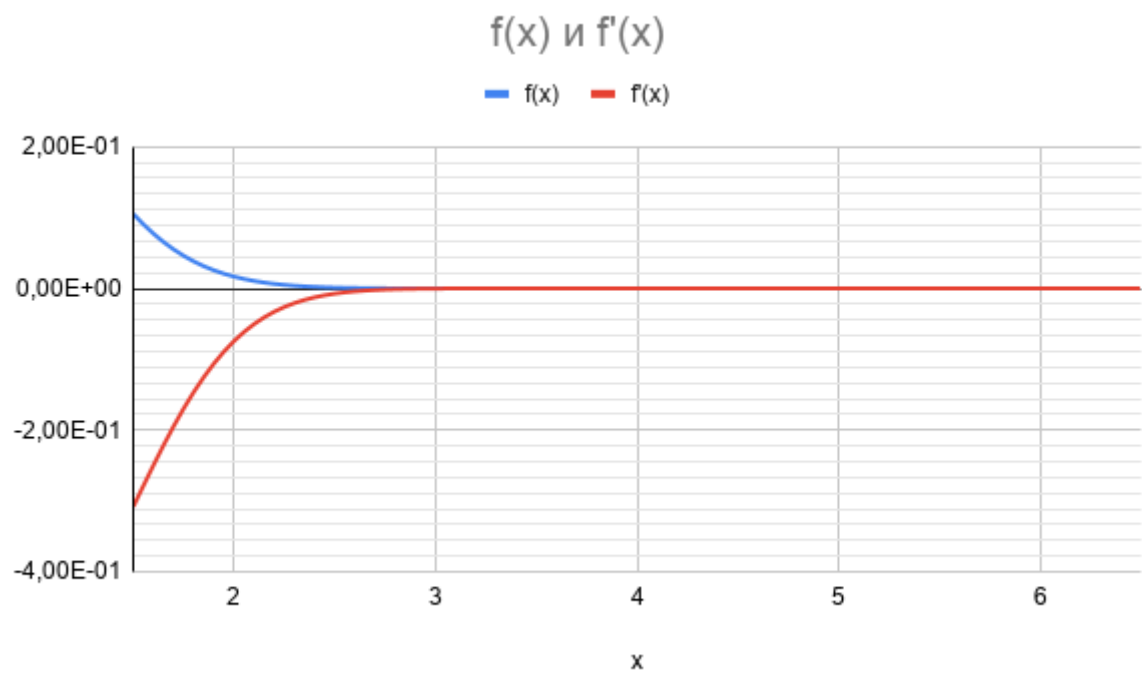


Рисунок 3 – Графік результатів функції 2 при $a=1$.

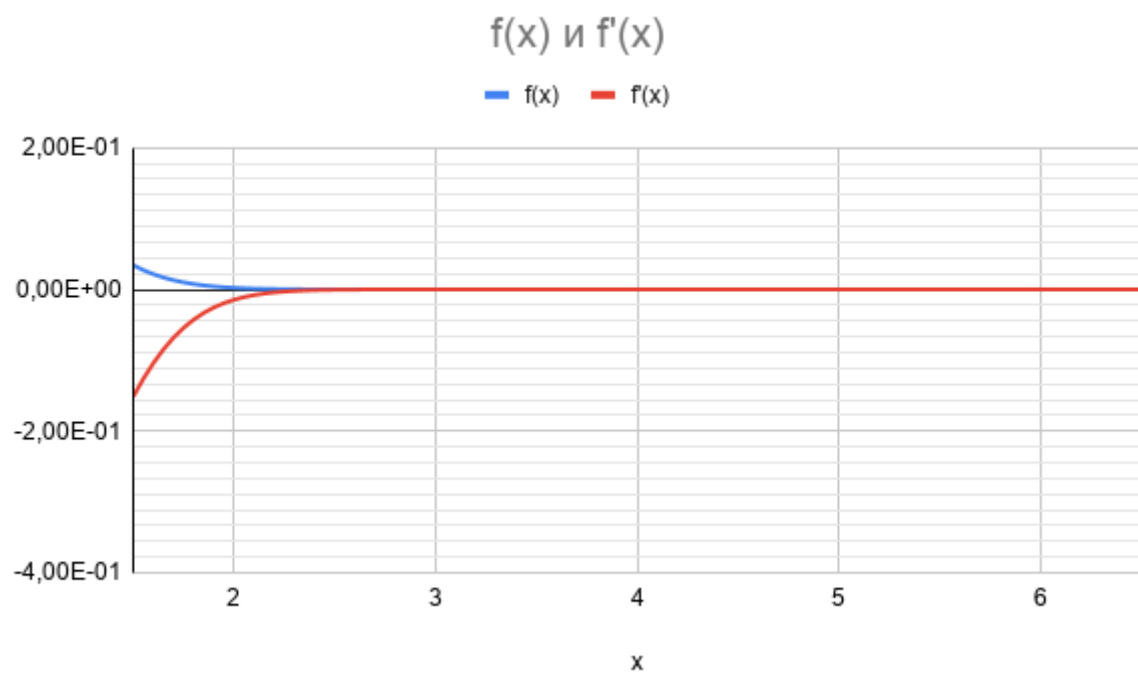


Рисунок 4 – Графік результатів функції 2 при $a=1,5$.

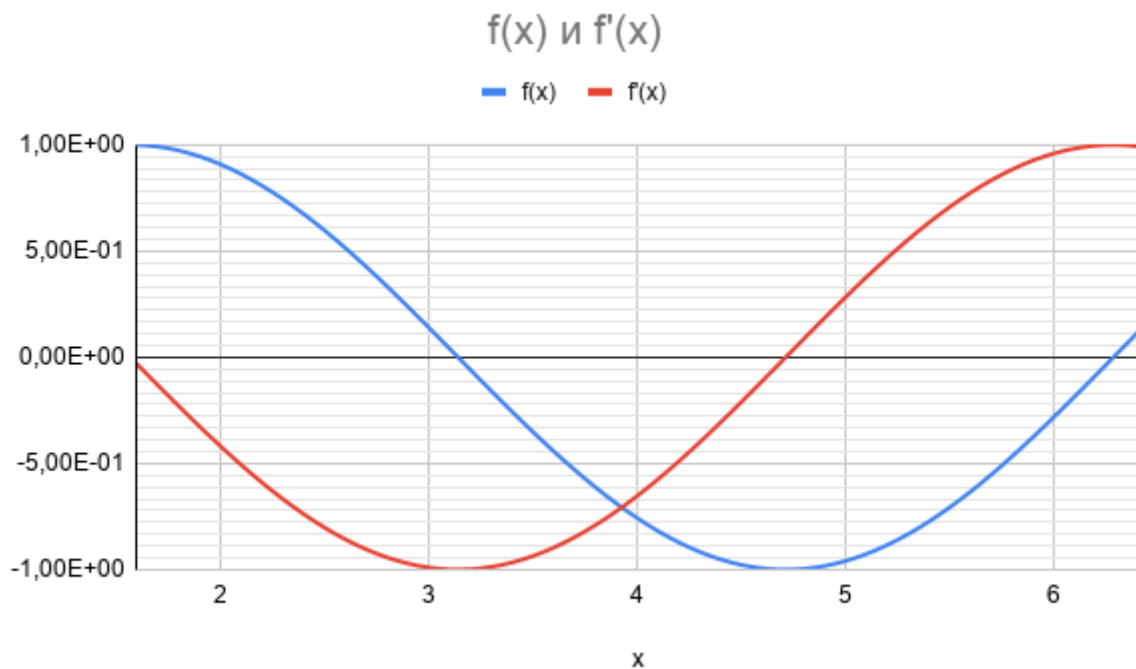


Рисунок 5 – Графік результатів функції 3.

Після виконання завдання 1, відповідно до умов завдання 2 було змінено структуру коду, розклавши один клас та його методи на різні класи за призначенням та функціоналом (див. Додаток Б).

Останнім кроком було додання відповідних до завдання 3 функцій шляхом додання нових класів або зміни старих (див. Додаток В). Результуюча програма дозволяє оперування розрахунками базових функцій без параметрів та з одним параметром.

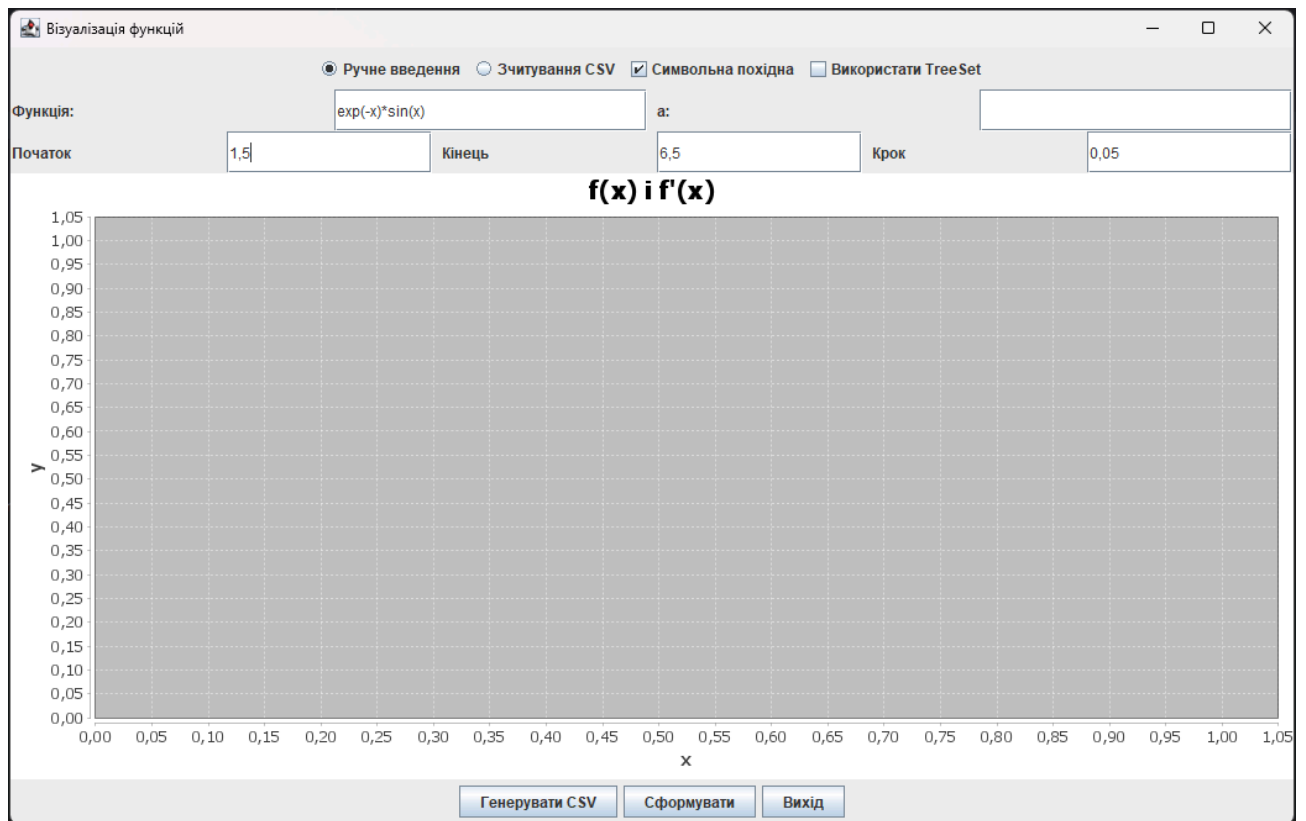


Рисунок 6 – Вигляд графічного інтерфейсу користувача.

Оперування програмою реалізується за допомогою полів для введення даних і кнопок з встановленим функціоналом:

- Поля розташовані на верхній частині інтерфейсу надають користувачу можливість вводити функцію та за потреби її параметр, місце для введення другорядних даних для обчислення функції (початок відрізка x , кінець та крок відповідно).
- Перемикач, розташований в верхній частині інтерфейсу визначає яким способом буде введена функція для обчислень (мануально/зчитана з файлу)
- Кнопка «Генерувати CSV» - створює файл csv для наглядної демонстрації підтримки табличного формату на прикладі тестової функції №3 ($\sin(x)$).
- Кнопка «Сформувати» відповідає за створення графіків функції та її похідної.
- Кнопки «Вихід» реалізує вихід з програми.

Нижче (див. Рисунок 7-8) наведено графіки тестових функцій, побудовані за допомогою програми та мануального введення функції.

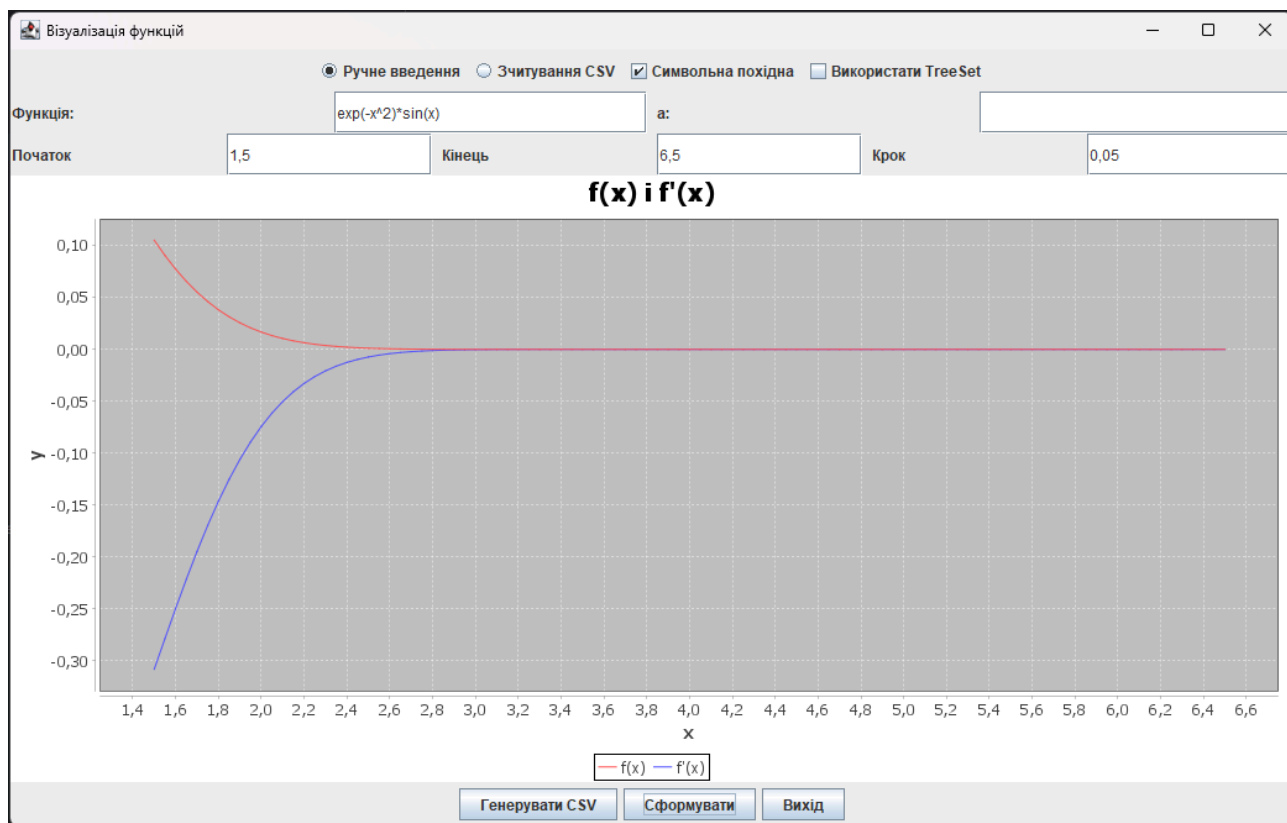


Рисунок 7 – Графік тестової функції №1 створений програмою.

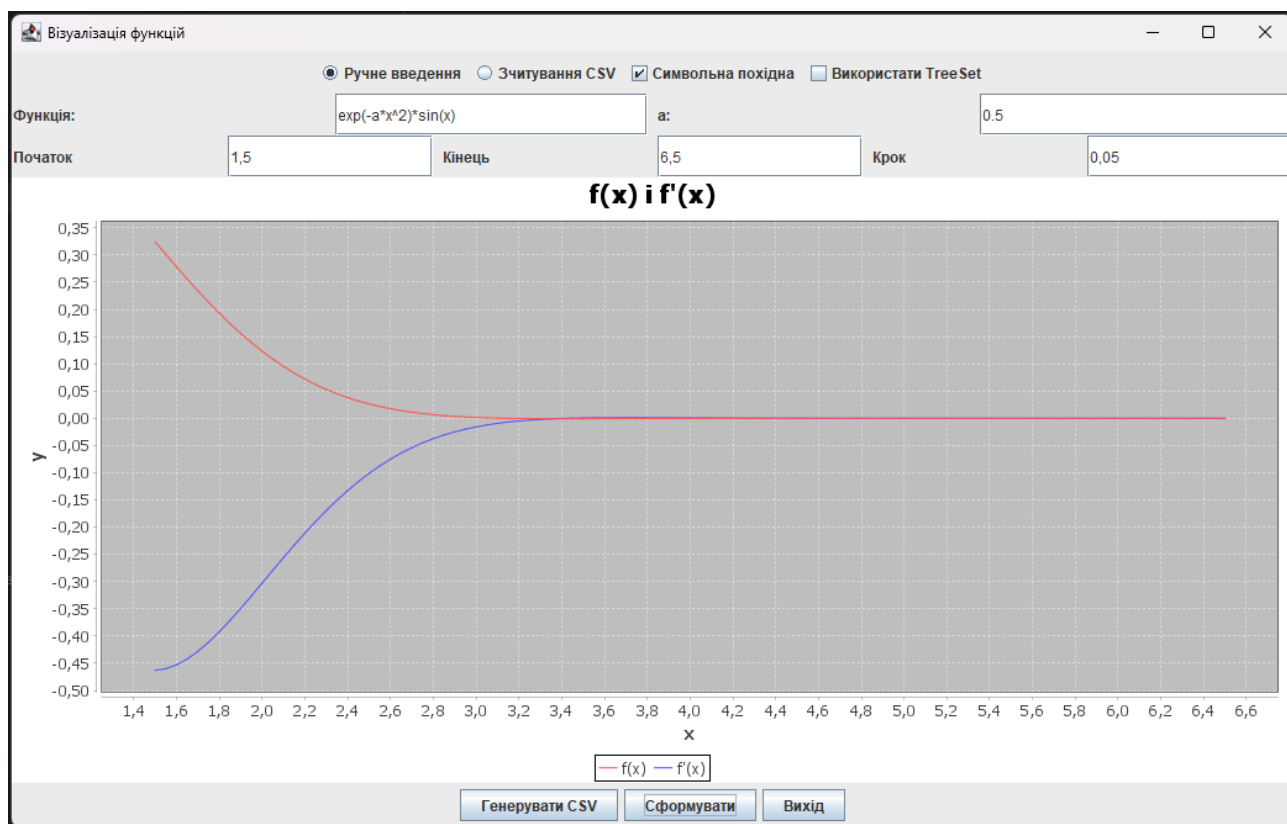


Рисунок 8 – Графік тестової функції №2 створений програмою.

На рисунку 9-11 продемонстровано роботу програми через метод зчитування з файлу CSV.

Файл	Редагувати	Переглянути
1,50000	0,9974949866	
1,55000	0,9997837642	
1,60000	0,9995736030	
1,65000	0,9968650285	
1,70000	0,9916648105	
1,75000	0,9839859469	
1,80000	0,9738476309	
1,85000	0,9612752030	
1,90000	0,9463000877	
1,95000	0,9289597150	
2,00000	0,9092974268	
2,05000	0,8873623686	
2,10000	0,8632093666	
2,15000	0,8368987908	
2,20000	0,8084964038	
2,25000	0,7780731969	
2,30000	0,7457052122	
2,35000	0,7114733528	
2,40000	0,6754631806	
2,45000	0,6377647021	
2,50000	0,5984721441	
2,55000	0,5576837174	

Рисунок 9 – Вигляд згенерованого файлу csv.

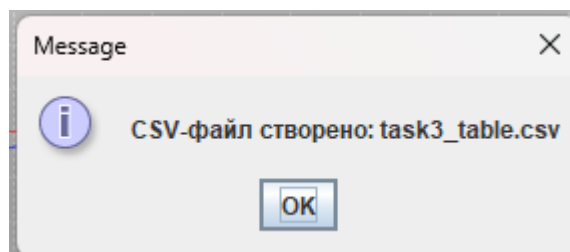


Рисунок 10 – Повідомлення програми про створення файлу csv.

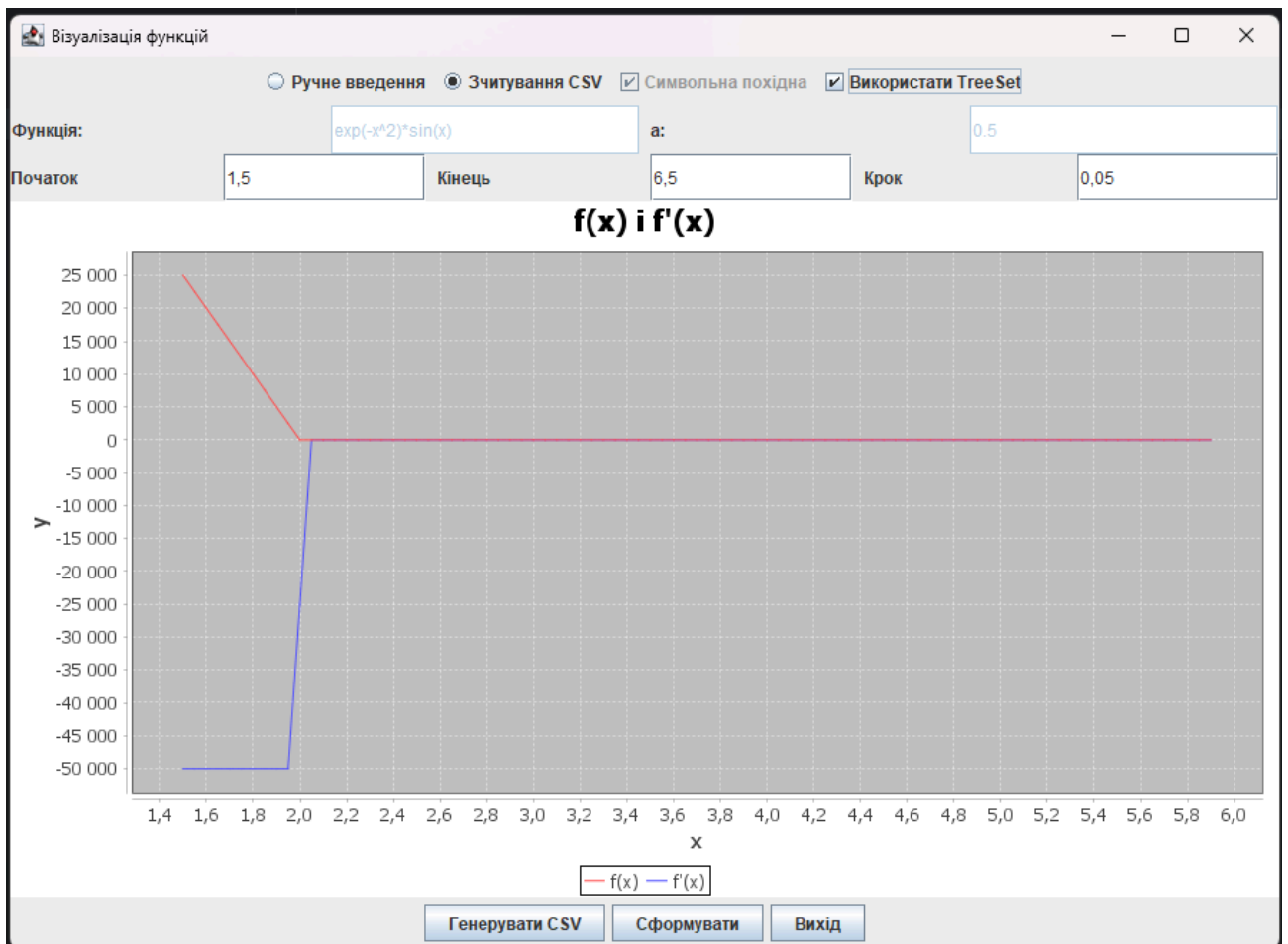


Рисунок 11 – Графік тестової функції №3 створений програмою.

Висновки:

У ході виконання розрахунково-графічної роботи №1 було реалізовано програму для обчислення значень функцій та їх похідних на заданому інтервалі.

На першому етапі було розроблено базову версію програми, яка забезпечує обчислення значень аналітичних та табличних функцій, а також чисельне знаходження похідної з використанням центральної різницевої формули. Результати обчислень зберігалися у текстові файли для подальшої обробки.

На другому етапі було виконано рефакторинг програмного коду. Програму було розділено на окремі класи відповідно до їх призначення (обчислення функцій, обчислення похідної, робота з файлами, генерація таблиць). Це дозволило підвищити читабельність, модульність та зручність супроводу коду.

На третьому етапі було розширено функціональні можливості програми. Зокрема, реалізовано графічний інтерфейс користувача для введення даних, побудови графіків функцій та їх похідних, а також підтримку зчитування даних із CSV-файлів. Додатково було реалізовано обчислення похідної як чисельним методом, так і за допомогою символічного диференціювання для аналітичних функцій.

У результаті виконання роботи було отримано програмний продукт, який дозволяє ефективно виконувати обчислення значень функцій, їх похідних та

візуалізувати результати у вигляді графіків. Отримані навички включають роботу з чисельними методами, обробку даних, побудову графічного інтерфейсу та організацію програмного коду відповідно до принципів об'єктно-орієнтованого програмування.

Додаток А. Вихідний код програми на етапі завдання №1 (Лістинг)

```
package task1;

import java.io.*;
import java.util.*;

public class MainTask1 {

    interface Evaluatable {
        double evalf(double x);
    }

    static class AnalyticalFunction implements Evaluatable {
        private double a;

        public AnalyticalFunction(double a) {
            this.a = a;
        }

        public double evalf(double x) {
            return Math.exp(-a * x * x) * Math.sin(x);
        }
    }

    static class Point2D {
        double x;
        double y;

        public Point2D(double x, double y) {
            this.x = x;
            this.y = y;
        }
    }

    static class TabulatedFunction implements Evaluatable {
        private final List<Point2D> data = new ArrayList<>();

        public void addPoint(double x, double y) {
            data.add(new Point2D(x, y));
        }

        public void readFromFile(String filename) throws IOException {
            data.clear();
            BufferedReader br = new BufferedReader(new FileReader(filename));
            String line;

            while ((line = br.readLine()) != null) {
                String[] parts = line.trim().split("\\s+");
                if (parts.length >= 2) {
                    double x = Double.parseDouble(parts[0].replace(',', ' '));
```

```

        double y = Double.parseDouble(parts[1].replace(',', ' '));
        addPoint(x, y);
    }
}

br.close();
data.sort(Comparator.comparingDouble(p -> p.x));
}

public double evalf(double x) {
    if (data.isEmpty()) {
        return Double.NaN;
    }

    if (data.size() == 1) {
        return data.get(0).y;
    }

    if (x < data.get(0).x || x > data.get(data.size() - 1).x) {
        return Double.NaN;
    }

    for (int i = 0; i < data.size() - 1; i++) {
        double x1 = data.get(i).x;
        double x2 = data.get(i + 1).x;

        if (x >= x1 && x <= x2) {
            double y1 = data.get(i).y;
            double y2 = data.get(i + 1).y;

            if (x2 == x1) {
                return y1;
            }

            return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
        }
    }

    return Double.NaN;
}

static class NumMethods {
    private static double derivativeStep(Evaluatable f, double x, double h) {
        double right = f.evalf(x + h);
        double left = f.evalf(x - h);

        if (Double.isNaN(right) || Double.isNaN(left) ||
            Double.isInfinite(right) || Double.isInfinite(left)) {
            return Double.NaN;
        }

        return (right - left) / (2.0 * h);
    }
}

public static double derivative(Evaluatable f, double x, double eps) {
    double h = 0.1;

    double d1 = derivativeStep(f, x, h);
    h *= 0.1;
    double d2 = derivativeStep(f, x, h);

    if (Double.isNaN(d1) || Double.isNaN(d2)) {
        return Double.NaN;
    }

    int maxIterations = 20;

```

```

        int count = 0;

        while (count < maxIterations) {
            h *= 0.1;
            double d3 = derivativeStep(f, x, h);

            if (Double.isNaN(d3) || Double.isInfinite(d3)) {
                return d2;
            }

            if (Math.abs(d3 - d2) >= Math.abs(d2 - d1)) {
                return d2;
            }

            if (Math.abs(d2 - d1) < eps) {
                return d2;
            }

            d1 = d2;
            d2 = d3;
            count++;
        }

        return d2;
    }
}

static String format(double value, int precision) {
    String pattern = "%. " + precision + "f";
    return String.format(Locale.US, pattern, value).replace('.', ',');
}

static void generateTable(String filename, double start, double end, double
step) throws IOException {
    PrintWriter writer = new PrintWriter(new FileWriter(filename));

    for (double x = start; x <= end + 1e-9; x += step) {
        writer.println(format(x, 5) + " " + format(Math.sin(x), 10));
    }

    writer.close();
}

static void computeAndSave(Evaluatable f,
                           String filename,
                           double start,
                           double end,
                           double step,
                           double eps) throws IOException {

    PrintWriter writer = new PrintWriter(new FileWriter(filename));
    writer.println("x\tf(x)\tf'(x)");

    for (double x = start; x <= end + 1e-9; x += step) {
        double fx = f.evalf(x);
        double dfx = NumMethods.derivative(f, x, eps);

        if (Double.isNaN(fx) || Double.isNaN(dfx) ||
            Double.isInfinite(fx) || Double.isInfinite(dfx)) {
            continue;
        }

        writer.println(
            format(x, 5) + "\t" +
                format(fx, 10) + "\t" +
                format(dfx, 10)
        );
    }
}

```

```

    }

    writer.close();
}

public static void main(String[] args) {
    double xStart = 1.5;
    double xEnd = 6.5;
    double step = 0.05;
    double eps = 1e-5;

    try {
        Evaluatable f1 = new AnalyticalFunction(1.0);
        computeAndSave(f1, "function1.txt", xStart, xEnd, step, eps);

        double[] aValues = {0.5, 1.0, 1.5};
        for (double a : aValues) {
            Evaluatable f2 = new AnalyticalFunction(a);
            computeAndSave(f2, "function2_a_" + a + ".txt", xStart, xEnd,
step, eps);
        }

        generateTable("table.txt", xStart, xEnd, step);

        TabulatedFunction f3 = new TabulatedFunction();
        f3.readFromFile("table.txt");
        computeAndSave(f3, "function3_table.txt", xStart, xEnd, step, eps);

        System.out.println("Готово");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Додаток Б. Вихідний код програми на етапі завдання №2 (Лістинг)

```

package task2;

import java.io.*;
import java.util.*;
import java.util.function.DoubleUnaryOperator;

public class MainTask2 {

    interface Function {
        double value(double x);
    }

    interface DerivativeCalculator {
        double derivative(Function function, double x, double eps);
    }

    static class AnalyticalFunction implements Function {
        private final DoubleUnaryOperator function;

        public AnalyticalFunction(DoubleUnaryOperator function) {
            this.function = function;
        }

        public double value(double x) {

```

```

        return function.applyAsDouble(x);
    }
}

static class DataPoint {
    private final double x;
    private final double y;

    public DataPoint(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }
}

static class TabulatedFunction implements Function {
    private final List<DataPoint> data;

    public TabulatedFunction(List<DataPoint> data) {
        this.data = new ArrayList<>(data);
        this.data.sort(Comparator.comparingDouble(DataPoint::getX));
    }

    public double value(double x) {
        if (data.isEmpty()) {
            return Double.NaN;
        }

        if (data.size() == 1) {
            return data.get(0).getY();
        }

        if (x < data.get(0).getX() || x > data.get(data.size() - 1).getX()) {
            return Double.NaN;
        }

        for (int i = 0; i < data.size() - 1; i++) {
            double x1 = data.get(i).getX();
            double x2 = data.get(i + 1).getX();

            if (x >= x1 && x <= x2) {
                double y1 = data.get(i).getY();
                double y2 = data.get(i + 1).getY();

                if (x2 == x1) {
                    return y1;
                }

                return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
            }
        }

        return Double.NaN;
    }
}

static class CentralDifferenceDerivative implements DerivativeCalculator {

    private double derivativeStep(Function function, double x, double h) {
        double left = function.value(x - h);

```

```

        double right = function.value(x + h);

        if (Double.isNaN(left) || Double.isNaN(right) ||
            Double.isInfinite(left) || Double.isInfinite(right)) {
            return Double.NaN;
        }

        return (right - left) / (2.0 * h);
    }

    public double derivative(Function function, double x, double eps) {
        double h = 0.1;

        double d1 = derivativeStep(function, x, h);
        h *= 0.1;
        double d2 = derivativeStep(function, x, h);

        if (Double.isNaN(d1) || Double.isNaN(d2)) {
            return Double.NaN;
        }

        int maxIterations = 20;
        int count = 0;

        while (count < maxIterations) {
            h *= 0.1;
            double d3 = derivativeStep(function, x, h);

            if (Double.isNaN(d3) || Double.isInfinite(d3)) {
                return d2;
            }

            if (Math.abs(d3 - d2) >= Math.abs(d2 - d1)) {
                return d2;
            }

            if (Math.abs(d2 - d1) < eps) {
                return d2;
            }

            d1 = d2;
            d2 = d3;
            count++;
        }

        return d2;
    }
}

static class TableGenerator {
    public void generateSinTable(String filename, double xStart, double xEnd,
double step) throws IOException {
        try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
            for (double x = xStart; x <= xEnd + 1e-9; x += step) {
                writer.println(format(x, 5) + " " + format(Math.sin(x), 10));
            }
        }
    }
}

static class TableFileReader {
    public TabulatedFunction read(String filename) throws IOException {
        List<DataPoint> points = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filename)))
{
            String line;

```



```

        while ((line = br.readLine()) != null) {
            String[] parts = line.trim().split("\\s+");
            if (parts.length >= 2) {
                double x = Double.parseDouble(parts[0].replace(',', ' '));
                double y = Double.parseDouble(parts[1].replace(',', ' '));
                points.add(new DataPoint(x, y));
            }
        }

        return new TabulatedFunction(points);
    }
}

static class ResultWriter {
    public void writeResults(String filename,
                             List<Double> xValues,
                             List<Double> yValues,
                             List<Double> derivatives) throws IOException {
        try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
            writer.println("x\tf(x)\tf'(x)");

            for (int i = 0; i < xValues.size(); i++) {
                writer.println(
                    format(xValues.get(i), 5) + "\t" +
                    format(yValues.get(i), 10) + "\t" +
                    format(derivatives.get(i), 10)
                );
            }
        }
    }
}

static class FunctionEvaluator {
    private final DerivativeCalculator derivativeCalculator;
    private final ResultWriter resultWriter;

    public FunctionEvaluator(DerivativeCalculator derivativeCalculator,
                             ResultWriter resultWriter) {
        this.derivativeCalculator = derivativeCalculator;
        this.resultWriter = resultWriter;
    }

    public void evaluate(Function function,
                         String filename,
                         double xStart,
                         double xEnd,
                         double step,
                         double eps) throws IOException {

        List<Double> xValues = new ArrayList<>();
        List<Double> yValues = new ArrayList<>();
        List<Double> derivatives = new ArrayList<>();

        for (double x = xStart; x <= xEnd + 1e-9; x += step) {
            double fx = function.value(x);
            double dfx = derivativeCalculator.derivative(function, x, eps);

            if (Double.isNaN(fx) || Double.isNaN(dfx) ||
                Double.isInfinite(fx) || Double.isInfinite(dfx)) {
                continue;
            }

            xValues.add(x);
            yValues.add(fx);
            derivatives.add(dfx);
        }
    }
}

```

```

    }

    resultWriter.writeResults(filename, xValues, yValues, derivatives);
}
}

static String format(double value, int precision) {
    String pattern = "%. " + precision + "f";
    return String.format(Locale.US, pattern, value).replace('.', ',');
}

public static void main(String[] args) {
    double xStart = 1.5;
    double xEnd = 6.5;
    double step = 0.05;
    double eps = 1e-5;

    try {
        DerivativeCalculator derivativeCalculator = new
CentralDifferenceDerivative();
        ResultWriter resultWriter = new ResultWriter();
        FunctionEvaluator evaluator = new
FunctionEvaluator(derivativeCalculator, resultWriter);
        TableGenerator tableGenerator = new TableGenerator();
        TableFileReader tableFileReader = new TableFileReader();

        Function f1 = new AnalyticalFunction(
            x -> Math.exp(-x * x) * Math.sin(x)
        );
        evaluator.evaluate(f1, "task2_function1.txt", xStart, xEnd, step,
eps);

        double[] aValues = {0.5, 1.0, 1.5};
        for (double a : aValues) {
            Function f2 = new AnalyticalFunction(
                x -> Math.exp(-a * x * x) * Math.sin(x)
            );
            evaluator.evaluate(f2, "task2_function2_a_" + a + ".txt", xStart,
xEnd, step, eps);
        }

        tableGenerator.generateSinTable("task2_table.txt", xStart, xEnd,
step);

        Function f3 = tableFileReader.read("task2_table.txt");
        evaluator.evaluate(f3, "task2_function3_table.txt", xStart, xEnd,
step, eps);

        System.out.println("Готово");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Додаток В. Вихідний код програми на етапі завдання №3 (Лістинг)

```

package task3;

import edu.hws.jcm.data.Expression;
import edu.hws.jcm.data.Variable;
import org.jfree.chart.ChartFactory;
import org.jfree.chart.ChartPanel;

```

```

import org.jfree.chart.JFreeChart;
import org.jfree.data.xy.XYSeries;
import org.jfree.data.xy.XYSeriesCollection;

import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.util.*;
import java.util.List;

public class MainTask3 extends JFrame {

    interface Function {
        double value(double x);
    }

    interface DerivativeCalculator {
        double derivative(Function function, double x, double eps);
    }

    static class AnalyticalFunction implements Function {
        private final String expressionText;
        private final Double aValue;
        private final edu.hws.jcm.data.Parser parser;
        private final Variable xVar;
        private Variable aVar;
        private final Expression expression;
        private final Expression symbolicDerivative;

        public AnalyticalFunction(String expressionText, Double aValue) {
            this.expressionText = expressionText;
            this.aValue = aValue;

            parser = new edu.hws.jcm.data.Parser();
            xVar = new Variable("x");
            parser.add(xVar);

            if (aValue != null) {
                aVar = new Variable("a");
                parser.add(aVar);
                aVar.setVal(aValue);
            }

            expression = parser.parse(expressionText);
            symbolicDerivative = expression.derivative(xVar);
        }

        public double value(double x) {
            xVar.setVal(x);
            if (aVar != null && aValue != null) {
                aVar.setVal(aValue);
            }
            return expression.getVal();
        }

        public double symbolicDerivative(double x) {
            xVar.setVal(x);
            if (aVar != null && aValue != null) {
                aVar.setVal(aValue);
            }
            return symbolicDerivative.getVal();
        }

        public String getExpressionText() {
            return expressionText;
        }
    }
}

```

```

static class DataPoint implements Comparable<DataPoint> {
    private final double x;
    private final double y;

    public DataPoint(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    public int compareTo(DataPoint other) {
        return Double.compare(this.x, other.x);
    }
}

static class TreeSetTabulatedFunction implements Function {
    private final TreeSet<DataPoint> data;

    public TreeSetTabulatedFunction(TreeSet<DataPoint> data) {
        this.data = data;
    }

    public double value(double x) {
        if (data.isEmpty()) {
            return Double.NaN;
        }

        List<DataPoint> points = new ArrayList<>(data);

        if (x < points.get(0).getX() || x > points.get(points.size() -
1).getX()) {
            return Double.NaN;
        }

        for (int i = 0; i < points.size() - 1; i++) {
            double x1 = points.get(i).getX();
            double x2 = points.get(i + 1).getX();

            if (x >= x1 && x <= x2) {
                double y1 = points.get(i).getY();
                double y2 = points.get(i + 1).getY();

                if (x2 == x1) {
                    return y1;
                }

                return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
            }
        }

        return Double.NaN;
    }
}

static class TreeMapTabulatedFunction implements Function {
    private final TreeMap<Double, Double> data;

    public TreeMapTabulatedFunction(TreeMap<Double, Double> data) {
        this.data = data;
    }
}

```

```

    }

    public double value(double x) {
        if (data.isEmpty()) {
            return Double.NaN;
        }

        if (data.containsKey(x)) {
            return data.get(x);
        }

        Map.Entry<Double, Double> lower = data.floorEntry(x);
        Map.Entry<Double, Double> higher = data.ceilingEntry(x);

        if (lower == null || higher == null) {
            return Double.NaN;
        }

        double x1 = lower.getKey();
        double y1 = lower.getValue();
        double x2 = higher.getKey();
        double y2 = higher.getValue();

        if (x2 == x1) {
            return y1;
        }

        return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
    }
}

static class NumericDerivativeCalculator implements DerivativeCalculator {

    private double derivativeStep(Function function, double x, double h) {
        double left = function.value(x - h);
        double right = function.value(x + h);

        if (Double.isNaN(left) || Double.isNaN(right) ||
            Double.isInfinite(left) || Double.isInfinite(right)) {
            return Double.NaN;
        }

        return (right - left) / (2.0 * h);
    }

    public double derivative(Function function, double x, double eps) {
        double h = 0.1;

        double d1 = derivativeStep(function, x, h);
        h *= 0.1;
        double d2 = derivativeStep(function, x, h);

        if (Double.isNaN(d1) || Double.isNaN(d2)) {
            return Double.NaN;
        }

        int maxIterations = 20;
        int count = 0;

        while (count < maxIterations) {
            h *= 0.1;
            double d3 = derivativeStep(function, x, h);

            if (Double.isNaN(d3) || Double.isInfinite(d3)) {
                return d2;
            }
        }
    }
}

```

```

        if (Math.abs(d3 - d2) >= Math.abs(d2 - d1)) {
            return d2;
        }

        if (Math.abs(d2 - d1) < eps) {
            return d2;
        }

        d1 = d2;
        d2 = d3;
        count++;
    }

    return d2;
}

}

static class SymbolicDerivativeCalculator implements DerivativeCalculator {
    public double derivative(Function function, double x, double eps) {
        if (function instanceof AnalyticalFunction analyticalFunction) {
            return analyticalFunction.symbolicDerivative(x);
        }
        return Double.NaN;
    }
}

static class CsvService {

    public void generateSinCsv(String filename, double xStart, double xEnd,
double step) throws IOException {
        try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
            writer.println("x,y");
            for (double x = xStart; x <= xEnd + 1e-9; x += step) {
                writer.println(format(x, 5) + "," + format(Math.sin(x), 10));
            }
        }
    }

    public TreeMapTabulatedFunction readTreeMapFunction(String filename)
throws IOException {
        TreeMap<Double, Double> data = new TreeMap<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filename)))
        {
            String line;
            boolean first = true;

            while ((line = br.readLine()) != null) {
                if (first) {
                    first = false;
                    if (line.toLowerCase().contains("x") &&
line.toLowerCase().contains("y")) {
                        continue;
                    }
                }

                String[] parts = line.split(",");
                if (parts.length >= 2) {
                    double x = Double.parseDouble(parts[0].trim().replace(',', '
'));
                    double y = Double.parseDouble(parts[1].trim().replace(',', '
'));
                    data.put(x, y);
                }
            }
        }
    }
}

```

```

        return new TreeMapTabulatedFunction(data);
    }

    public TreeSetTabulatedFunction readTreeSetFunction(String filename)
    throws IOException {
        TreeSet<DataPoint> data = new TreeSet<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filename)))
        {
            String line;
            boolean first = true;

            while ((line = br.readLine()) != null) {
                if (first) {
                    first = false;
                    if (line.toLowerCase().contains("x") &&
line.toLowerCase().contains("y")) {
                        continue;
                    }
                }

                String[] parts = line.split(",");
                if (parts.length >= 2) {
                    double x = Double.parseDouble(parts[0].trim().replace(',', '
'));
                    double y = Double.parseDouble(parts[1].trim().replace(',', '
'));
                    data.add(new DataPoint(x, y));
                }
            }

            return new TreeSetTabulatedFunction(data);
        }

        static class ResultWriter {
            public void writeResults(String filename,
                                    List<Double> xValues,
                                    List<Double> yValues,
                                    List<Double> derivatives) throws IOException {
                try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
                    writer.println("x\tf(x)\tf'(x)");
                    for (int i = 0; i < xValues.size(); i++) {
                        writer.println(
                            format(xValues.get(i), 5) + "\t" +
                                format(yValues.get(i), 10) + "\t" +
                                format(derivatives.get(i), 10)
                            );
                    }
                }
            }
        }

        private final JRadioButton manualInputRadio = new JRadioButton("Ручне
введення", true);
        private final JRadioButton csvInputRadio = new JRadioButton("Зчитування CSV");
        private final JTextField functionField = new JTextField("exp(-x)*sin(x)");
        private final JTextField aField = new JTextField("");
        private final JTextField xStartField = new JTextField("1,5");
        private final JTextField xEndField = new JTextField("6,5");
        private final JTextField stepField = new JTextField("0,05");
        private final JCheckBox symbolicDerivativeCheck = new JCheckBox("Символьна
похідна", true);
        private final JCheckBox treeSetCheck = new JCheckBox("Використати TreeSet");
        private final JButton generateCsvButton = new JButton("Генерувати CSV");
        private final JButton buildButton = new JButton("Сформувати");

```

```

private final JButton exitButton = new JButton("Вихід");
private final ChartPanel chartPanel;
private final CsvService csvService = new CsvService();
private final ResultWriter resultWriter = new ResultWriter();
private static final String CSV_FILE = "task3_table.csv";

public MainTask3() {
    setTitle("Візуалізація функцій");
    setSize(950, 650);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLayout(new BorderLayout());

    JPanel topPanel = new JPanel(new GridLayout(3, 1));

    ButtonGroup group = new ButtonGroup();
    group.add(manualInputRadio);
    group.add(csvInputRadio);

    JPanel radioPanel = new JPanel();
    radioPanel.add(manualInputRadio);
    radioPanel.add(csvInputRadio);
    radioPanel.add(symbolicDerivativeCheck);
    radioPanel.add(treeSetCheck);
    topPanel.add(radioPanel);

    JPanel functionPanel = new JPanel(new GridLayout(1, 4, 8, 8));
    functionPanel.add(new JLabel("Функція:"));
    functionPanel.add(functionField);
    functionPanel.add(new JLabel("a:"));
    functionPanel.add(aField);
    topPanel.add(functionPanel);

    JPanel rangePanel = new JPanel(new GridLayout(1, 6, 8, 8));
    rangePanel.add(new JLabel("Початок"));
    rangePanel.add(xStartField);
    rangePanel.add(new JLabel("Кінець"));
    rangePanel.add(xEndField);
    rangePanel.add(new JLabel("Крок"));
    rangePanel.add(stepField);
    topPanel.add(rangePanel);

    add(topPanel, BorderLayout.NORTH);

    XYSeriesCollection dataset = new XYSeriesCollection();
    JFreeChart chart = ChartFactory.createXYLineChart(
        "f(x) i f'(x)",
        "x",
        "y",
        dataset
    );

    chartPanel = new ChartPanel(chart);
    add(chartPanel, BorderLayout.CENTER);

    JPanel bottomPanel = new JPanel();
    bottomPanel.add(generateCsvButton);
    bottomPanel.add(buildButton);
    bottomPanel.add(exitButton);
    add(bottomPanel, BorderLayout.SOUTH);

    manualInputRadio.addActionListener(e -> switchMode());
    csvInputRadio.addActionListener(e -> switchMode());

    generateCsvButton.addActionListener(e -> generateCsv());
    buildButton.addActionListener(e -> buildPlot());
    exitButton.addActionListener(e -> System.exit(0));
}

```



```

        switchMode();
    }

    private void switchMode() {
        boolean manual = manualInputRadio.isSelected();
        functionField.setEnabled(manual);
        aField.setEnabled(manual);
        symbolicDerivativeCheck.setEnabled(manual);
    }

    private void generateCsv() {
        try {
            double xStart = parseNumber(xStartField.getText());
            double xEnd = parseNumber(xEndField.getText());
            double step = parseNumber(stepField.getText());

            csvService.generateSinCsv(CSV_FILE, xStart, xEnd, step);
            JOptionPane.showMessageDialog(this, "CSV-файл створено: " + CSV_FILE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Помилка генерації CSV: " +
ex.getMessage());
        }
    }

    private void buildPlot() {
        try {
            double xStart = parseNumber(xStartField.getText());
            double xEnd = parseNumber(xEndField.getText());
            double step = parseNumber(stepField.getText());
            double eps = 1e-5;

            Function function;
            DerivativeCalculator derivativeCalculator;

            if (manualInputRadio.isSelected()) {
                String expr = functionField.getText().trim();
                Double aValue = null;

                if (expr.contains("a")) {
                    String text = aField.getText().trim();
                    if (text.isEmpty()) {
                        throw new IllegalArgumentException("Для функції з
параметром a введіть значення a.");
                    }
                    aValue = parseNumber(text);
                }

                AnalyticalFunction analyticalFunction = new
AnalyticalFunction(expr, aValue);
                function = analyticalFunction;

                if (symbolicDerivativeCheck.isSelected()) {
                    derivativeCalculator = new SymbolicDerivativeCalculator();
                } else {
                    derivativeCalculator = new NumericDerivativeCalculator();
                }
            } else {
                File file = new File(CSV_FILE);
                if (!file.exists()) {
                    generateCsv();
                }

                if (treeSetCheck.isSelected()) {
                    function = csvService.readTreeSetFunction(CSV_FILE);
                } else {
                    function = csvService.readTreeMapFunction(CSV_FILE);
                }
            }
        }
    }

```

```

        derivativeCalculator = new NumericDerivativeCalculator();
    }

    XYSeries functionSeries = new XYSeries("f(x)");
    XYSeries derivativeSeries = new XYSeries("f'(x)");

    List<Double> xValues = new ArrayList<>();
    List<Double> yValues = new ArrayList<>();
    List<Double> dValues = new ArrayList<>();

    for (double x = xStart; x <= xEnd + 1e-9; x += step) {
        double y = function.value(x);
        double dy = derivativeCalculator.derivative(function, x, eps);

        if (Double.isNaN(y) || Double.isNaN(dy) ||
            Double.isInfinite(y) || Double.isInfinite(dy)) {
            continue;
        }

        functionSeries.add(x, y);
        derivativeSeries.add(x, dy);

        xValues.add(x);
        yValues.add(y);
        dValues.add(dy);
    }

    XYSeriesCollection dataset = new XYSeriesCollection();
    dataset.addSeries(functionSeries);
    dataset.addSeries(derivativeSeries);
    chartPanel.getChart().getXYPlot().setDataset(dataset);

    String outputName = manualInputRadio.isSelected() ?
"task3_manual_output.txt" : "task3_csv_output.txt";
    resultWriter.writeResults(outputName, xValues, yValues, dValues);

    JOptionPane.showMessageDialog(this, "Результати побудовано і збережено
y " + outputName);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Помилка: " + ex.getMessage());
    }
}

static double parseNumber(String text) {
    return Double.parseDouble(text.trim().replace(',', '.', ''));
}

static String format(double value, int precision) {
    String pattern = "%. " + precision + "f";
    return String.format(Locale.US, pattern, value).replace('.', ',');
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new MainTask3().setVisible(true));
}
}

```